

DP Computer Science: B 4.1.2 Evaluate Linked Lists (HL Only)

1. Core Concepts & Learning Objectives

Lesson 1: Introduction to Singly Linked Lists

Learning Intentions

By the end of this lesson, students will be able to:

- Describe the fundamental structure of a **node** in a linked list (**data and pointer**)
- Explain the overall structure of a **singly linked list** (head pointer, sequence of nodes, null termination)
- Compare the basic properties of singly linked lists and **arrays** (dynamic vs. static size, memory allocation)
- Demonstrate **diagrammatically** the traversal of a singly linked list
- Demonstrate **diagrammatically** the process of **inserting** a new node into a singly linked list (at the head, at the tail, and in the middle)

Key Vocabulary

Term	Definition
Node	A fundamental building block of a linked list; contains data and a pointer to the next node
Head	A pointer to the first node in the linked list
Tail	A pointer to the last node in the linked list
Null	A special value indicating the end of the list (no next node)
Pointer	A variable that stores the memory address of another node
Singly Linked List (SLL)	A linked list where each node points to the next node only
Traversal	The process of visiting each node in a linked list
Dynamic	The size of the data structure can change at runtime

ATL Elements (Lesson 1)

Skill	Description
Thinking	Comparing data structures (arrays vs. SLLs); analysing steps in diagrammatic operations (traversal, insertion)
Communication	Explaining the structure of SLLs and operations using diagrams
Self-Management	Organising diagrammatic steps logically

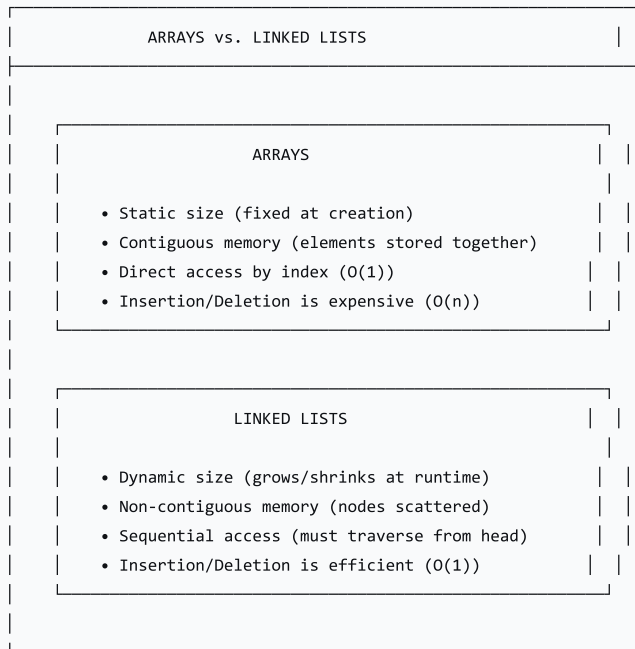
2. Introduction: Why Different Data Structures?

Data Structures Overview

Data structures are ways of organising and storing data in a computer so that it can be used efficiently.

Arrays vs. Linked Lists

text



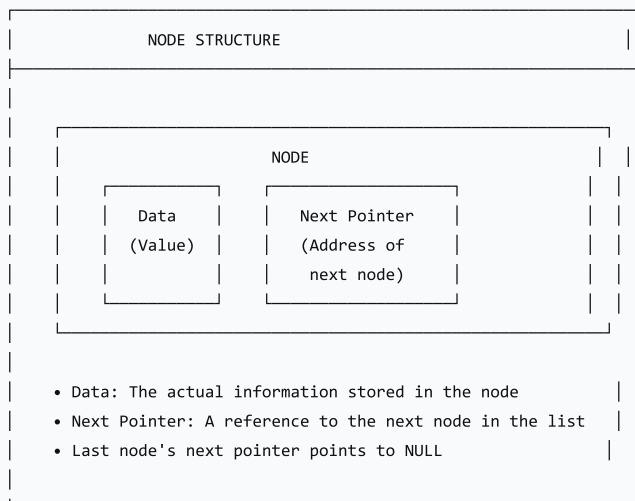
The Scavenger Hunt Analogy

"A linked list is like a scavenger hunt: each clue tells you where to find the next clue. You must follow the clues in order to reach the end."

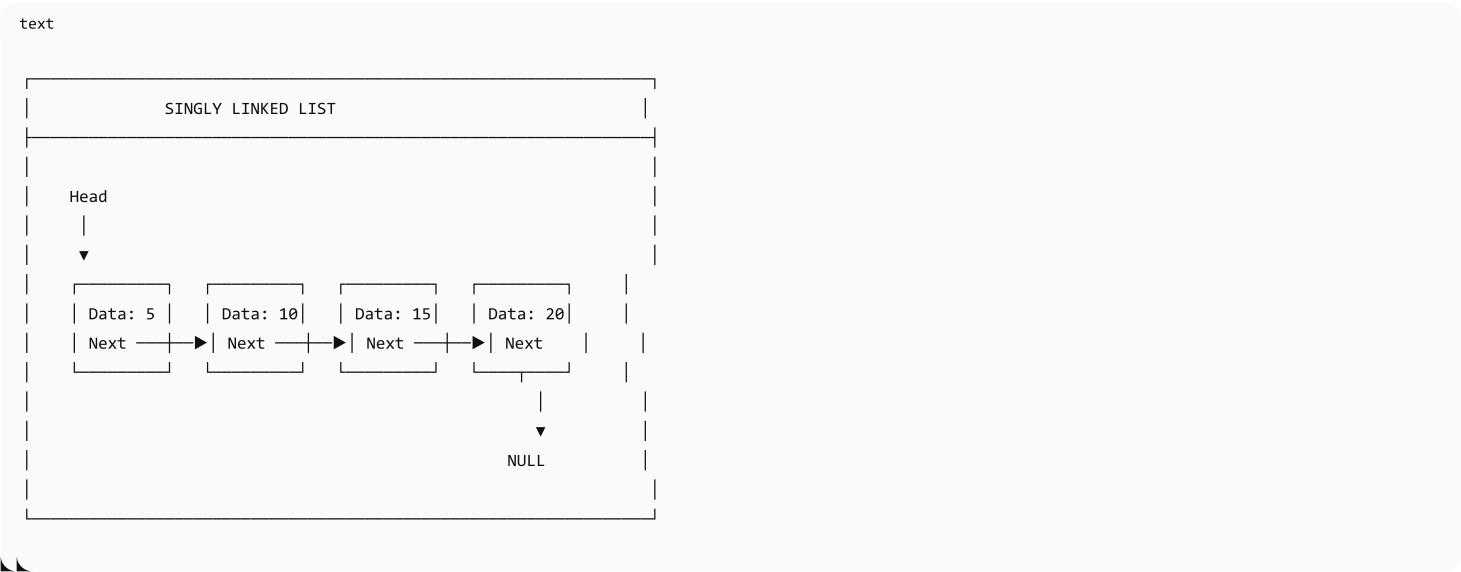
3. Singly Linked List (SLL) Structure

Node Structure

text



Singly Linked List Diagram



Key Components of an SLL

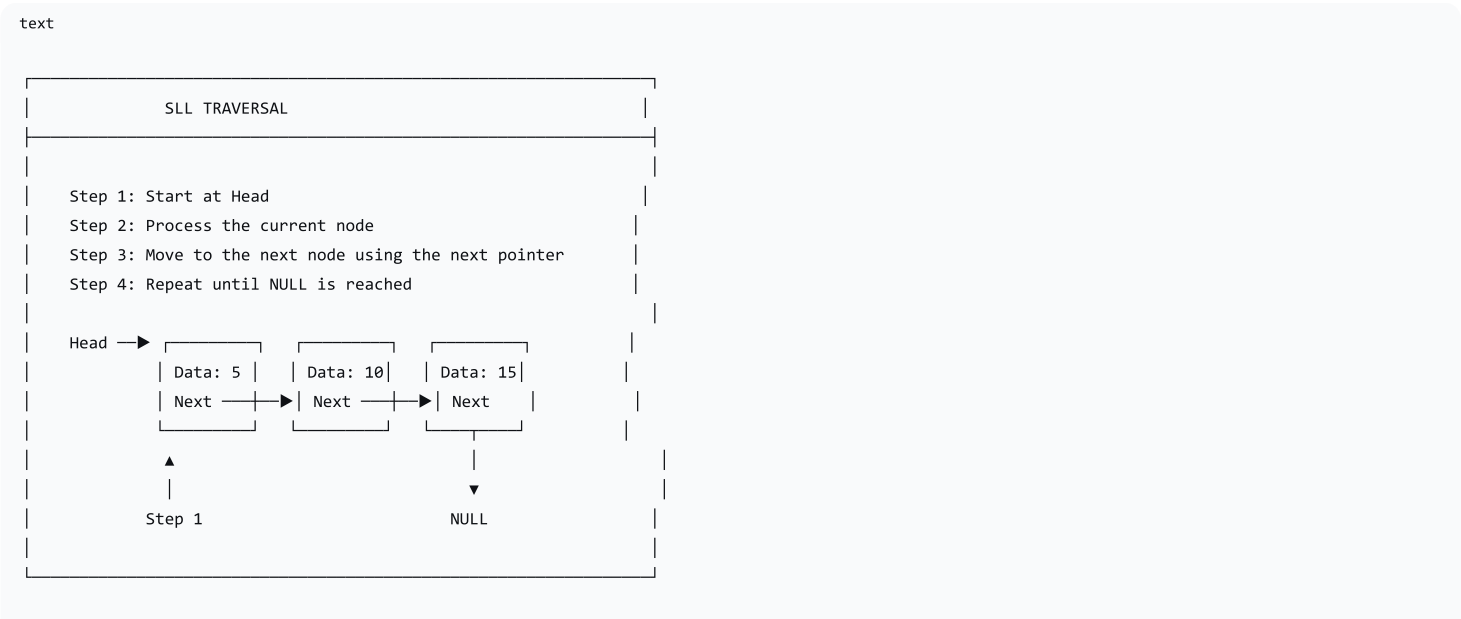
Component	Description
Node	Contains data and a pointer to the next node
Head	Points to the first node in the list
Null	Marks the end of the list (last node's next is NULL)

4. SLL Traversal

What is Traversal?

Traversal is the process of visiting each node in a linked list, starting from the head and following the next pointers until NULL is reached.

Diagrammatic Traversal

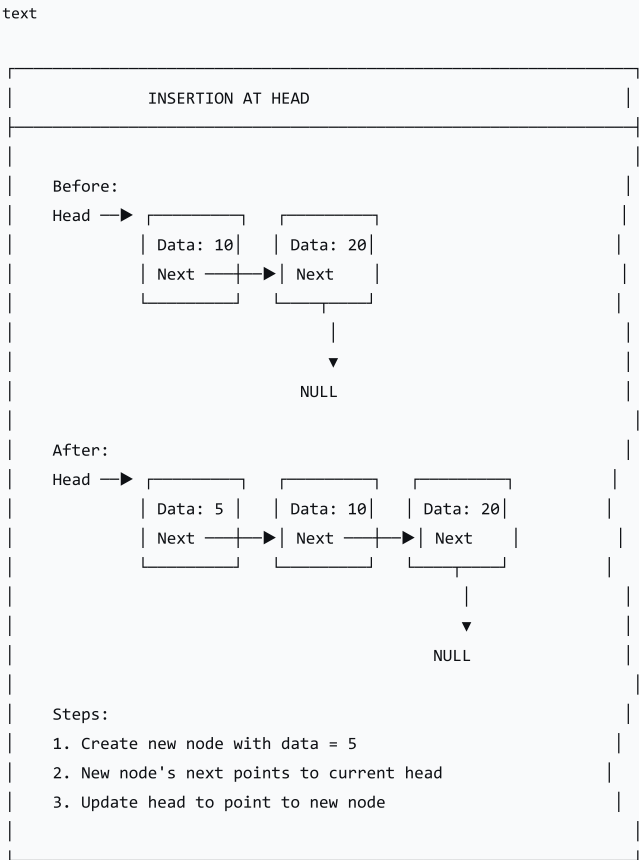


Traversal Algorithm

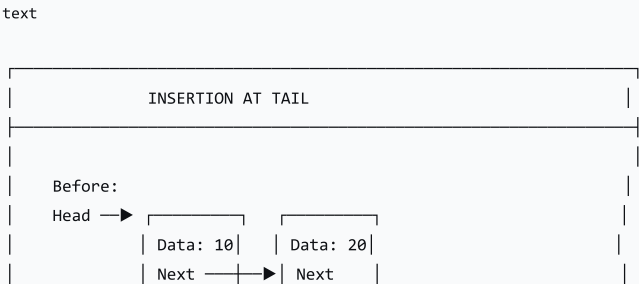
```
python
# Python: Traversing a Singly Linked List
def traverse(head):
    current = head
    while current is not None:
        print(current.data)
        current = current.next
```

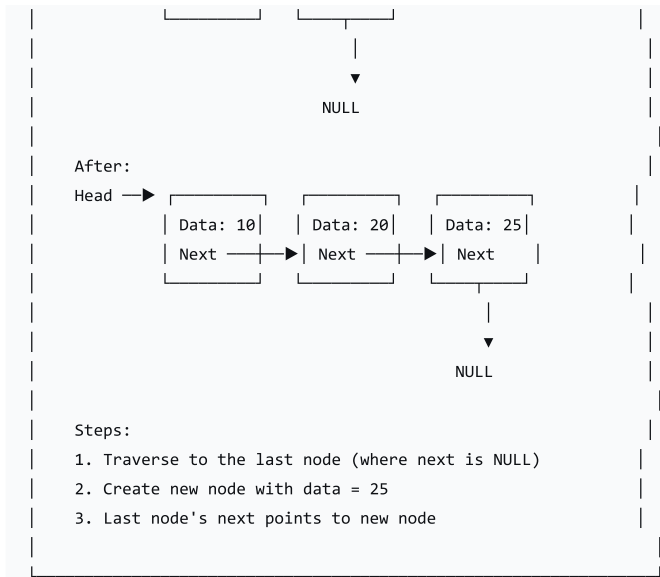
5. SLL Insertion (Diagrammatic)

Insertion at the Head

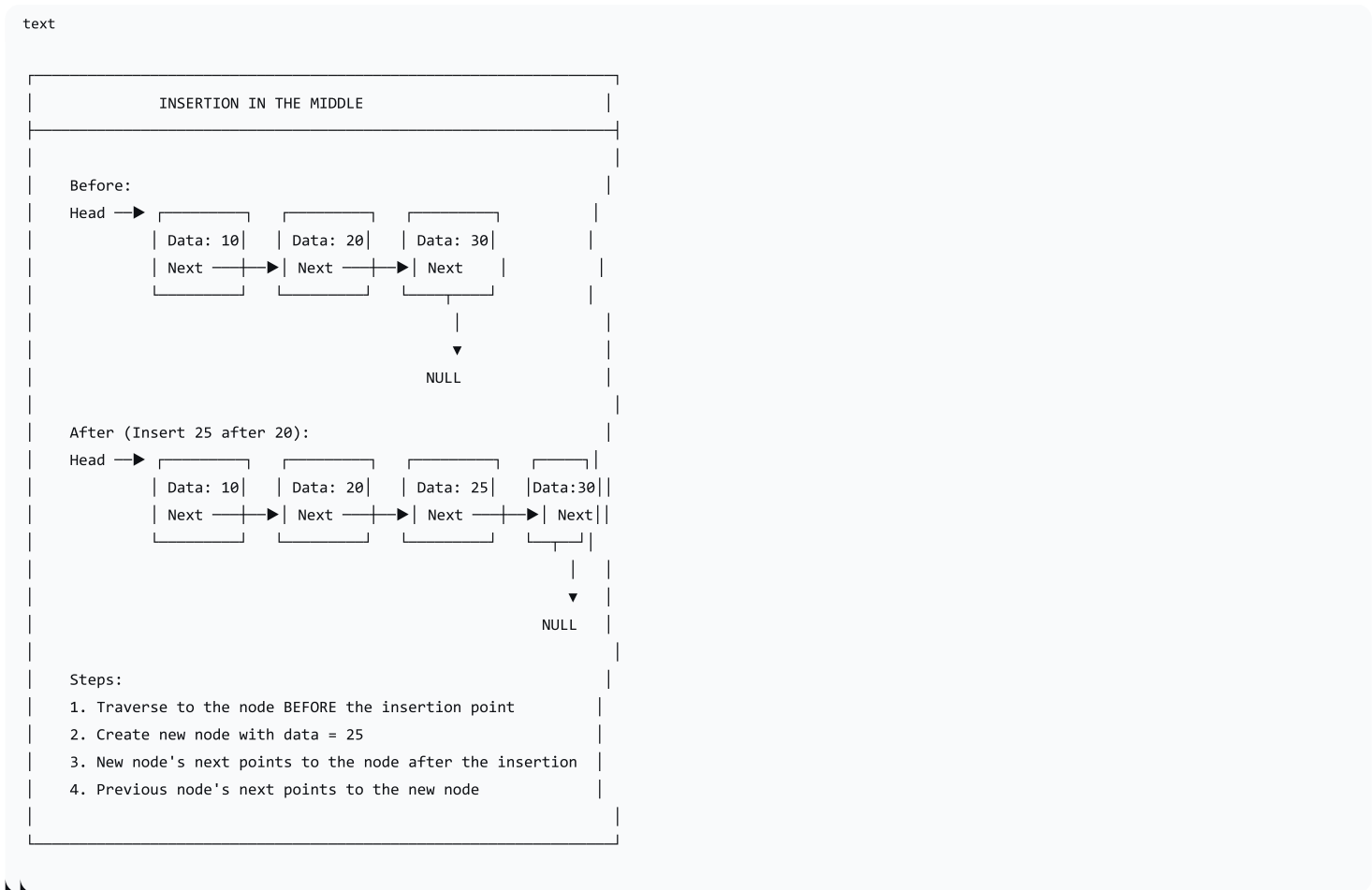


Insertion at the Tail





Insertion in the Middle



6. Lesson 2: SLL Deletion, Search, and Doubly Linked Lists

Learning Intentions

By the end of this lesson, students will be able to:

- Demonstrate diagrammatically the process of **deleting** a node from a singly linked list (at the head, at the tail, and in the middle)
- Demonstrate diagrammatically the process of **searching** for a value in a singly linked list
- Describe the fundamental structure of a **node in a doubly linked list** (data, previous pointer, and next pointer)
- Explain the overall structure of a **doubly linked list** (head pointer, sequence of nodes, tail pointer, null termination at both ends)
- Demonstrate diagrammatically basic operations on a **doubly linked list**
- Explain an **advantage** of using a doubly linked list compared to a singly linked list

7. SLL Deletion (Diagrammatic)

Deletion at the Head

text

DELETION AT HEAD

Before:

Head →

Data: 5	Data: 10	Data: 15
Next →	Next →	Next

↓
NULL

After:

Head →

Data: 10	Data: 15
Next →	Next

↓
NULL

Steps:

1. Store current head in a temporary variable

2. Update head to point to the second node

3. Delete the old head node

Deletion in the Middle

text

DELETION IN THE MIDDLE

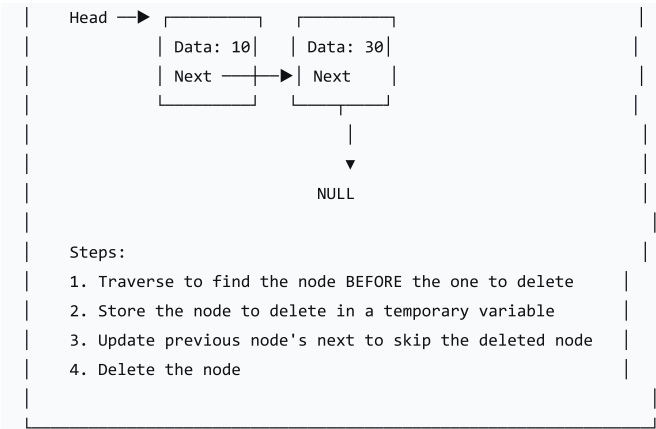
Before:

Head →

Data: 10	Data: 20	Data: 30
Next →	Next →	Next

↓
NULL

After (Delete 20):



8. Doubly Linked Lists (DLL)

Node Structure

text

DOUBLY LINKED LIST NODE

NODE

Prev
Pointer

Data
(Value)

Next
Pointer

- Prev Pointer: Points to the previous node
- Data: The actual information stored
- Next Pointer: Points to the next node
- First node's prev is NULL
- Last node's next is NULL

Doubly Linked List Diagram

text

DOUBLY LINKED LIST

Head
↓

Prev Data: 5 Next →	Prev → Data: 10 Next →	Prev → Data: 15 Next
---------------------------	------------------------------	----------------------------

NULL ← ← ←



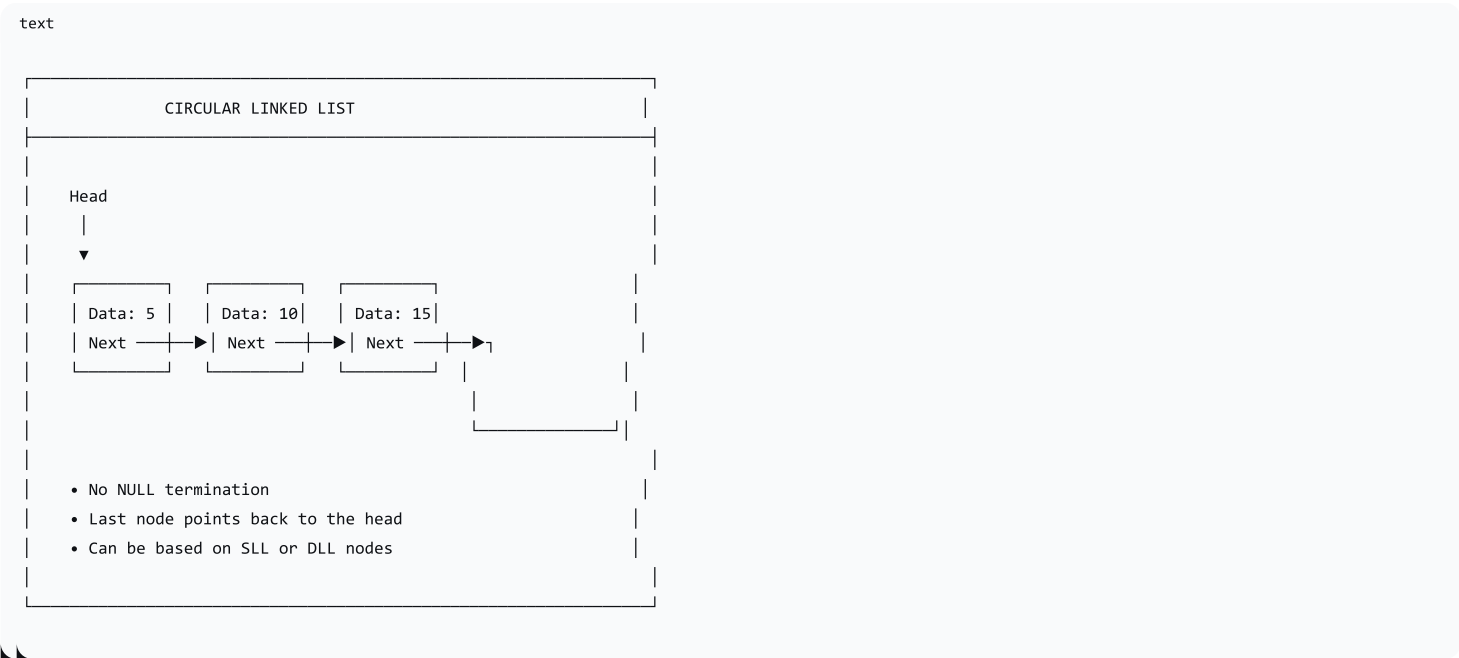
Advantage of DLL

A doubly linked list allows traversal in both directions (forward and backward). Deletion of a node is easier because you have direct access to both the previous and next nodes.

9. Circular Linked Lists (CLL)

Structure

A circular linked list is a linked list where the last node points back to the head, forming a loop. There is no NULL termination.



10. Performance Comparison (Time Complexity)

Operation	Array	SLL	DLL	CLL
Access by Index	O(1)	O(n)	O(n)	O(n)
Search by Value	O(n)	O(n)	O(n)	O(n)
Insert at Head	O(n)	O(1)	O(1)	O(1)
Insert at Tail	O(1)	O(n)	O(1)	O(n)
Insert in Middle	O(n)	O(n)	O(n)	O(n)
Delete at Head	O(n)	O(1)	O(1)	O(1)
Delete at Tail	O(1)	O(n)	O(1)	O(n)
Delete in Middle	O(n)	O(n)	O(1)	O(n)

11. Memory Comparison

Data Structure	Memory per Node	Overhead
Array	Data only	None
Singly Linked List	Data + 1 pointer	One pointer per node
Doubly Linked List	Data + 2 pointers	Two pointers per node
Circular Linked List	Data + 1 or 2 pointers	Same as SLL or DLL

Trade-offs

text

MEMORY TRADE-OFFS

Arrays:

✔ No overhead

✘ Fixed size

SLL:

✔ Dynamic size

✘ One pointer per node

✘ Cannot traverse backwards

DLL:

✔ Can traverse backwards

✔ Easier deletion

✘ Two pointers per node

12. Worksheets

Worksheet	Topic
Worksheet 1	SLL structure, traversal, insertion
Worksheet 2	SLL deletion, search, DLL structure and operations
Worksheet 3	CLL structure, performance comparison, use cases

13. Summary: Key Takeaways

Structure	Key Feature	Best For
Array	Direct access, static size	Fixed-size data, frequent random access
Singly Linked List	One-way traversal, dynamic size	Simple lists, frequent insertions/deletions at head
Doubly Linked List	Two-way traversal	Frequent insertions/deletions, backward traversal

Structure	Key Feature	Best For
Circular Linked List	No NULL termination	Round-robin scheduling, continuous loops

text

KEY REMINDERS
✔ Node = Data + Pointer ✔ SLL = one-way traversal ✔ DLL = two-way traversal ✔ CLL = last node points to head ✔ Arrays = $O(1)$ access, $O(n)$ insert/delete ✔ Linked lists = $O(n)$ access, $O(1)$ insert/delete (at head)

"The choice of data structure depends on the specific requirements—speed, memory, and flexibility all play a role."